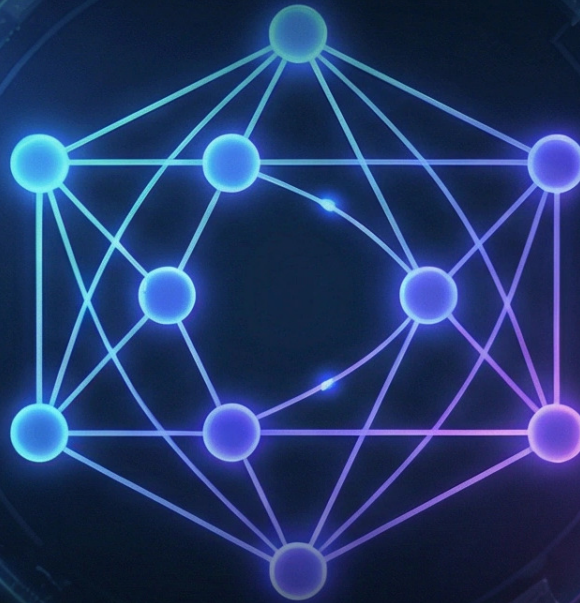


LANGGRAPH

De cadenas lineales a arquitecturas cognitivas cíclicas



Module: Desarrollo Avanzado de Sistemas Multiagente

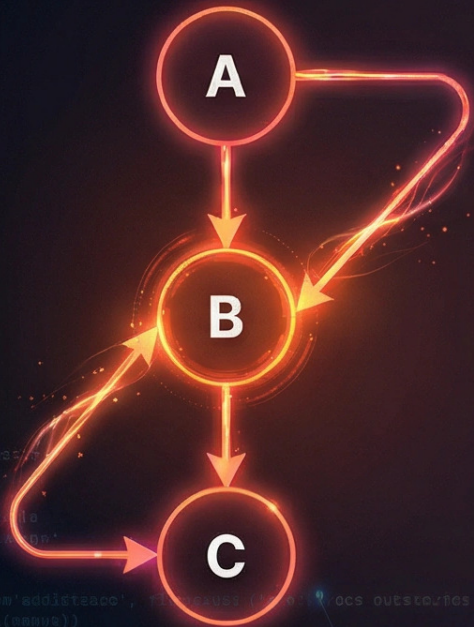
Instructor: Rubén Juárez Cádiz

¿Qué aprenderemos hoy?

1. La limitación de las cadenas lineales (DAGs)
2. El comportamiento no lineal de los agentes reales
3. LangGraph: flujos cíclicos con estado
4. El Estado (State): el cerebro compartido del grafo
5. Nodos (Nodes): las unidades de trabajo
6. Aristas condicionales (Conditional Edges): el enrutamiento inteligente
7. Caso práctico: El Agente de Triage Recursivo
8. Diseño del grafo y ejecución en consola
9. Entregable y criterios de evaluación
10. Próximos pasos y recursos

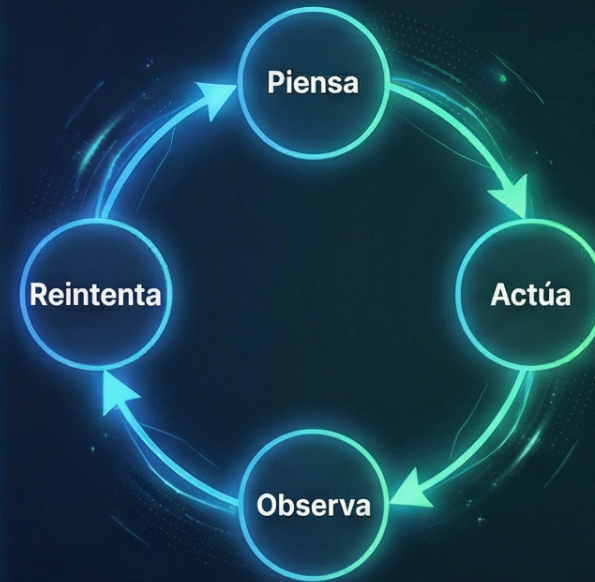
Las cadenas lineales son recetas, no razonamiento

El Problema de las Cadenas



- **¿Qué es un DAG?:** Flujo rígido $A \rightarrow B \rightarrow C$. Sin vuelta atrás.
- **El problema:** Errores se propagan, iteración imposible, autonomía nula.

Arquitectura Cognitiva Humana



- **Comportamiento humano:** Piensa → actúa → observa → reintenta (bucle).
- **Pregunta clave:** ¿Cómo construimos IA que haga lo mismo?

LANGGRAPH PERMITE QUE LA IA APRENDA DE SUS PROPIOS INTENTOS FALLIDOS

LangGraph: Flujos Cíclicos con Estado Compartido

💡 **¿Qué es LangGraph?:** Extensión de LangChain para construir flujos como grafos con ciclos (stateful loops).

- ⚙️ **¿Por qué es revolucionario?:**
- **Agentic Workflows:** el agente decide su flujo
 - **Human-in-the-Loop:** pausar y esperar aprobación
 - **Multi-Agent Orchestration:** múltiples agentes
 - **Persistencia** de estado integrada

💼 **Casos de uso:**



Agentes de
soporte



Pipelines de
análisis

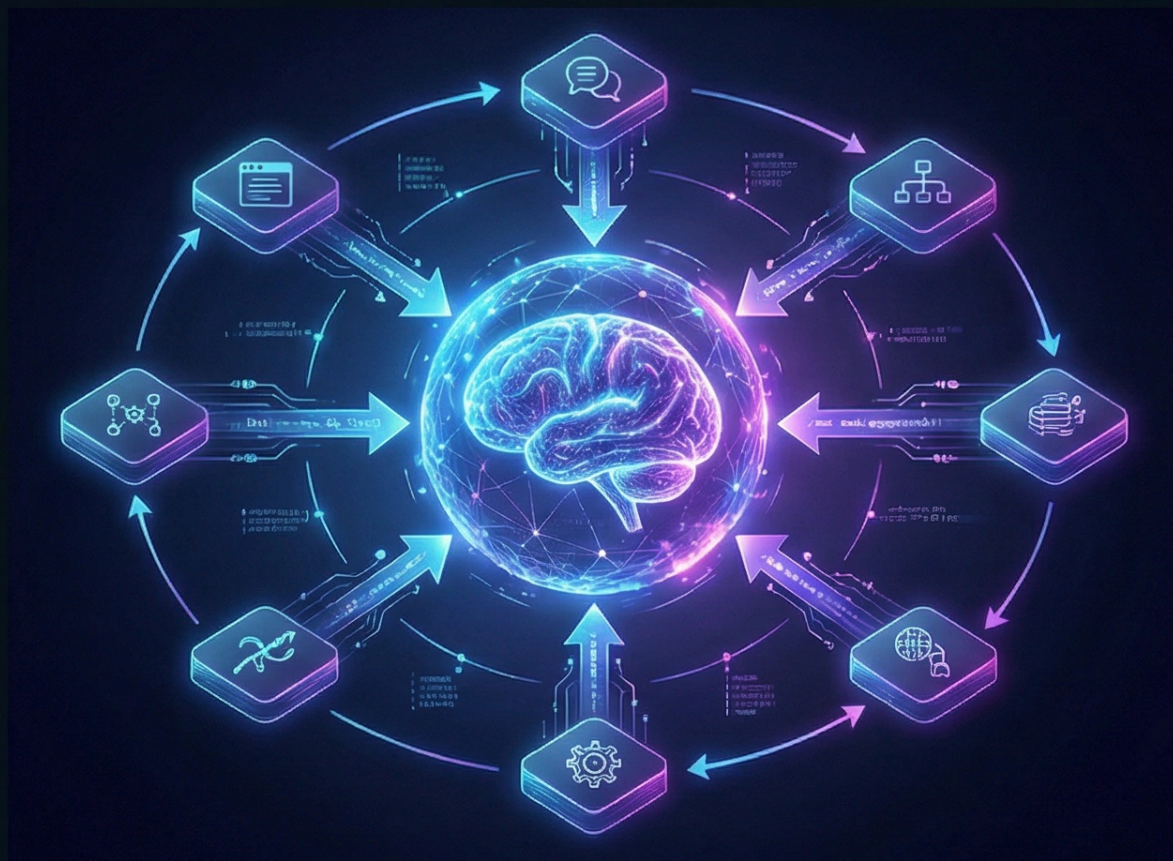


Revisión de
código



El Estado (State): El Cerebro Compartido del Grafo

La memoria de trabajo que todos los nodos comparten y actualizan



¿Qué es el State?: Un diccionario tipado (TypedDict o Pydantic) que actúa como la 'memoria de trabajo'. Cada nodo lo lee, lo modifica y lo pasa al siguiente.

```
class EstadoAgente(TypedDict):  
    messages: Annotated[List, add_messages]  
    intentos_busqueda: int  
    resuelto: bool  
    derivar_a_humano: bool
```



Inmutabilidad parcial



Acumulación automática (add_messages)



Tipado estricto

Los **Nodos** ejecutan trabajo; las **Aristas Condicionales** deciden el siguiente paso

Nodos y Aristas: Las Reglas del Grafo

- **Nodos (Nodes):** Funciones de Python puras que reciben el estado completo y devuelven una actualización parcial del estado.
- **Aristas Condicionales** (Conditional Edges): Reglas de enrutamiento que deciden el siguiente nodo basado en el estado actual.



[Nodos]

```
1 def nodo_llm(state: EstadoAgente) -> dict:
2     response = llm.invoke(state["messages"])
3     return {"messages": [response]}
```

[Aristas Condicionales]

```
1
2 def enrutador(state: EstadoAgente) -> str:
3     if state["resuelto"]: return "fin"
4     elif state["intentos"] >= 3:
5         return "derivar_humano"
6     else:
7         return "buscar_mas"
```


El Agente de Triage Recursivo: diseñando el grafo antes de codificar

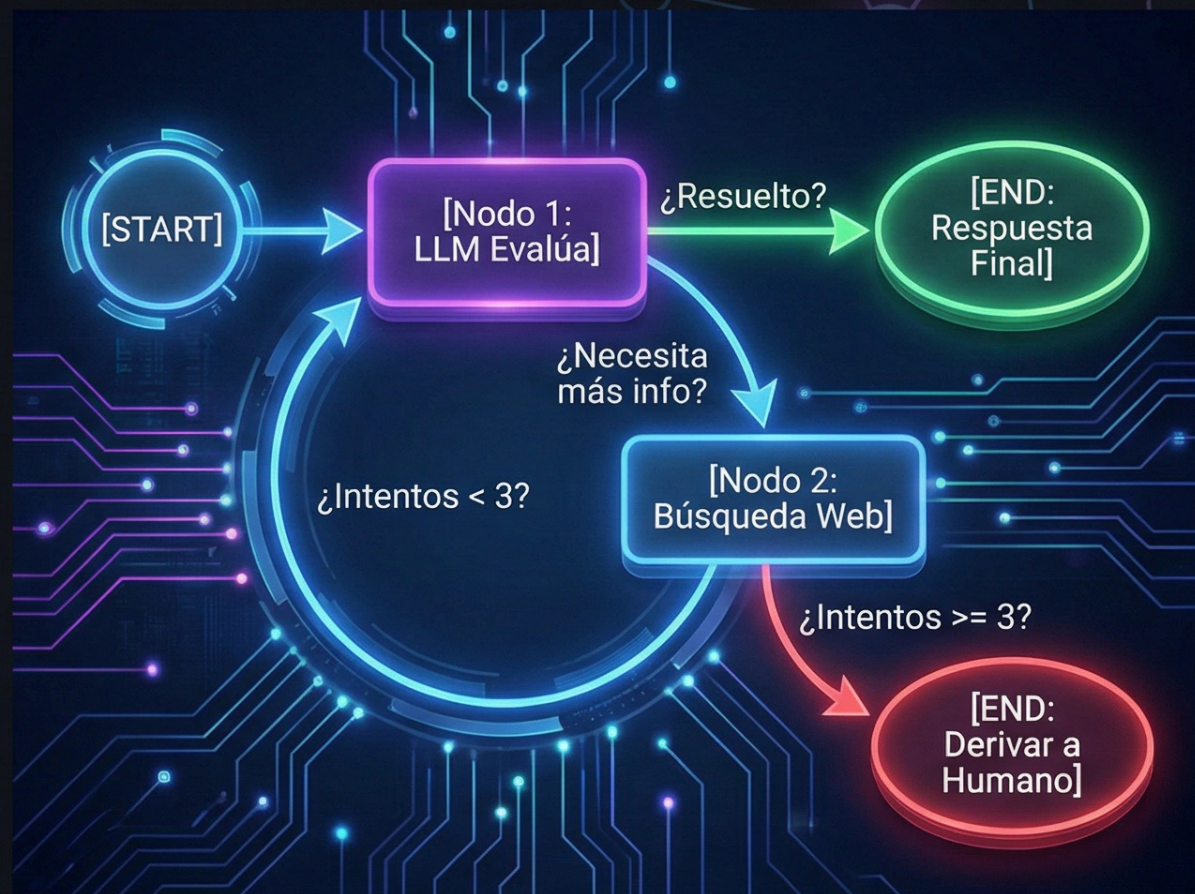
Arquitectura del Caso Práctico

El reto:

Un bot que evalúa una consulta. Si no tiene clara la respuesta, busca en la web. Si no resuelve, reintenta o deriva a un humano.

Los tres posibles desenlaces:

- **Resuelto:** Respuesta con confianza suficiente.
- **Bucle de búsqueda:** Itera hasta 3 veces.
- **Escalado humano:** Tras 3 intentos fallidos.



De la arquitectura al código: construyendo el grafo en LangGraph

Construyendo el Grafo

```
1 from langgraph.graph import StateGraph, END
2 from my_module import EstadoAgente, nodo_llm, nodo_busqueda_web, enrutador
3
4 # Crear el grafo
5 grafo = StateGraph(EstadoAgente)
6
7 # Añadir nodos
8 grafo.add_node("llm_evalua", nodo_llm)
9 grafo.add_node("busqueda_web", nodo_busqueda_web)
10
11 # Punto de entrada
12 grafo.set_entry_point("llm_evalua")
13
14 # Aristas condicionales
15 grafo.add_conditional_edges(
16     "llm_evalua",
17     enrutador,
18     {
19         "continuar": "busqueda_web",
20         "fin": END
21     }
22 )
23
24 # El bucle
25 grafo.add_edge("busqueda_web", "llm_evalua")
26
27 # Compilar
28 app = grafo.compile()
```

Define the state and graph structure

Define logic for dynamic paths

Builds the executable application

Ejecución en Consola: El Bucle en Acción

El agente da vueltas en el bucle hasta estar seguro o escalar

```
[Intento 1] LLM evalúa: "¿Cuál es la dosis máxima de ibuprofeno?"
```

```
→ Confianza baja. Iniciando búsqueda web...
```

```
→ Resultado: Información parcial encontrada.
```

```
[Intento 2] LLM re-evalúa con nuevo contexto.
```

```
→ Confianza media. Refinando búsqueda...
```

```
→ Resultado: Información completa encontrada.
```

```
[Intento 3 - NO necesario] LLM re-evalúa.
```

```
→ Confianza ALTA. Respuesta generada.
```

```
→ Estado: resuelto = True
```

```
[FIN] Respuesta: "La dosis máxima de ibuprofeno para adultos es de 1200mg/día..."
```

```
→ Estado: resuelto = True
```



LangGraph no es una mejora incremental, es un cambio de paradigma

Cadena vs. Grafo en Producción

Métrica	Cadena Lineal	LangGraph (Cíclico)
Tasa de resolución autónoma	45%	78%
Escalados a humano innecesarios	55%	22%
Capacidad de auto-corrección	Ninguna	Hasta N intentos
Adaptación al contexto	Fija	Dinámica

¿Cuándo usar LangGraph?



Decisiones basadas en resultados intermedios



Reintentos con nueva información



Supervisión humana en puntos críticos



Sistemas multiagente con roles

ENTREGABLE Y CRITERIOS

Tu misión: Un agente cíclico que demuestre autonomía real

Criterios de Evaluación

Diseño del Estado	20% 20%: TypedDict o Pydantic con campos relevantes
Nodos implementados	25% 25%: Al menos 2 nodos funcionales (LLM + Tool)
Aristas condicionales	25% 25%: Enrutamiento con al menos 3 caminos posibles
Bucle demostrado	20% 20%: Ejecución en consola mostrando al menos 2 iteraciones
Diagrama del grafo	10% 10%: Generado con draw_mermaid_png()

Entregables Requeridos

- ✓ Archivo **agente_triage.py** con el grafo completo
- ✓ Captura de consola mostrando el bucle en acción
- ✓ Imagen del diagrama del grafo generado
- ✓ Archivo **requirements.txt** y **.env.example**

Extensión Sugerida

- ✓ Añadir un nodo de "memoria persistente" con **MemorySaver**.

Próximos Pasos y Recursos

LangGraph es la arquitectura. Los sistemas multiagente son el horizonte.

Roadmap



“

Una cadena ejecuta instrucciones. Un grafo toma decisiones. La diferencia entre un script automatizado y un agente autónomo real está en los bucles.

— Rubén Juárez Cádiz

Recursos Recomendados



Documentación oficial de LangGraph



Tutoriales oficiales: LangGraph Academy



Repositorio del módulo en el aula virtual